

RxGuard: Personalized Drug Interaction Checker — Requirements, Datasets & Architecture

Sep 26, 2026 · @Sathya

Overview and scope

RxGuard tells a clinician or caregiver how risky it is for one specific patient to take two medicines together, as a 0–10 score with the reasons and cited label text.

Problem. Standard interaction checkers give the same answer for every patient. Real risk depends on the person: the same pair can be minor for a healthy 30-year-old and dangerous for a 68-year-old with liver disease who drinks daily.

What it does. The user enters two medicine names (brand or generic) and a patient ID. RxGuard normalises the names, looks up the interaction severity, pulls the patient's age, conditions, labs and alcohol use from their health record, adjusts the risk with transparent rules, and explains the result in plain language using FDA label text.

Key design principle. Severity comes only from a curated database or a trained ML model. The LLM never decides severity or score; it only explains, using retrieved label passages.

Version 1 scope

- Exactly 2 medicines per check.
- Adult patients (18+).
- Synthetic patients only (Synthea); no real patient data.
- Out of scope: dosing advice, drug–food interactions, children and pregnancy.

Stakeholders

The primary users are pharmacists and clinicians checking a patient's medicines; patients benefit indirectly.

Stakeholder	Group	What they need from RxGuard
Pharmacists	Primary user	A fast, patient-specific check before dispensing
Doctors and nurses	Primary user	Risk flags with reasons before prescribing or giving a medicine
Caregivers of elderly patients	Primary user	Plain-language warnings for a parent on many medicines
Patients	Indirect beneficiary	Fewer harmful combinations
Hospitals and clinics	Secondary	Fewer adverse drug events and readmissions
Regulators and clinical safety reviewers	Oversight	Traceable severity sources and cited explanations
NLM (RxNorm), DDInter, openFDA, Synthea	Data provider	Correct attribution and fair API use
College guide and evaluators	Project	Clear scope, working demo, measured results

Not for: self-diagnosis or dosing decisions. Every result shows "Clinical decision support only. Not medical advice."

Input, processing and output

The user types three fields; everything else about the patient is fetched from their record.

Input

Field	Type	Example
drug_a	Text, brand or generic	Tylenol
drug_b	Text, brand or generic	Warfarin
patient_id	Synthetic FHIR patient ID	P-1042

Fetches from the record: age, conditions, eGFR (kidney function), ALT (liver enzyme), alcohol use in drinks per day.

Processing steps

1. **Normalise names** (RxNorm): Tylenol → acetaminophen. Unknown name → "Did you mean...?" suggestions.
2. **Base severity** (DDInter): Major, Moderate, Minor or None. Pair not in the database → ML model predicts it, labelled "predicted" with a confidence value.
3. **Patient context** (FHIR server with Synthea data).
4. **Risk score** from the rules below.
5. **Explanation** (RAG over FDA labels + LLM), citing the label sections used.

Scoring rules

Rule	Points
Base severity: Major / Moderate / Minor / None	6 / 4 / 2 / 0
Each patient condition listed as a caution for either drug	+2
Age 65 or over	+1
eGFR below 60 and a drug is kidney-cleared	+2
Liver disease or ALT above 3× normal, and a drug is liver-metabolised	+2
Alcohol 3 or more drinks/day and acetaminophen involved	+2

Score capped at 10. Bands: 0–3 Low, 4–6 Moderate, 7–10 High. The weights are a starting design; tuning them against the evaluation scenarios is part of the project.

Output

```
{
  "drugs": ["acetaminophen", "warfarin"],
  "base_severity": "Moderate",
  "severity_source": "DDInter",
  "risk_score": 9,
  "risk_level": "High",
  "score_breakdown": [
    {"reason": "Base interaction: Moderate", "points": 4},
    {"reason": "Chronic liver disease + acetaminophen", "points": 2},
    {"reason": "Alcohol 4 drinks/day + acetaminophen", "points": 2},
    {"reason": "Age 68", "points": 1}
  ],
  "mechanism": "Regular acetaminophen use can increase warfarin's effect (raised INR), raising bleeding risk.",
  "affected_systems": ["blood clotting", "liver"],
  "monitoring": ["Check INR more often", "Review daily acetaminophen total", "Review alcohol use"],
  "citations": ["FDA label: warfarin, Drug Interactions", "FDA label: acetaminophen, Warnings"],
  "disclaimer": "Clinical decision support only. Not medical advice."
}
```

The same pair for patient P-2210 (age 30, no conditions, no alcohol) scores 4, Moderate. The UI renders the output as a severity gauge, a points breakdown, a body map of affected systems and cited explanation text.

Functional requirements

RxGuard has 18 functional requirements across seven modules; the 12 marked Must form the minimum demo.

ID	Module	Requirement	Priority
FR-01	Input	Accept two medicine names (brand, generic or misspelled) and a patient ID, with autocomplete	Must

ID	Module	Requirement	Priority
FR-02	Normaliser	Map each name to an RxNorm ingredient ID; offer up to 3 suggestions when no exact match	Must
FR-03	Normaliser	Split combination products into ingredients (e.g. a cold tablet → each active ingredient) and check each	Should
FR-04	Interaction lookup	Return the DDInter severity for the ingredient pair	Must
FR-05	ML predictor	Predict severity for pairs missing from DDInter, with a confidence value and a "predicted" label	Must
FR-06	Patient context	Fetch age, active conditions, latest eGFR and ALT, and alcohol use from the FHIR server	Must
FR-07	Patient context	Show which record fields were missing and score without them, flagged	Should
FR-08	Rules engine	Compute the 0-10 score and band using the scoring rules table	Must
FR-09	Rules engine	Return a line-by-line breakdown of points and reasons	Must
FR-10	Rules engine	Load rules and drug flags (kidney-cleared, liver-metabolised, condition cautions) from config files, not code	Should
FR-11	RAG explainer	Retrieve the relevant FDA label passages (interactions, warnings, contraindications) for both drugs	Must
FR-12	RAG explainer	Generate mechanism, affected systems and monitoring points using only retrieved passages, each cited	Must
FR-13	RAG explainer	Reject any generated sentence whose citation is not in the retrieved set	Must
FR-14	UI	Severity gauge, points breakdown and cited explanation on one results screen	Must
FR-15	UI	Body map highlighting affected systems	Should

ID	Module	Requirement	Priority
FR-16	UI	Side-by-side compare: same drug pair for two patients	Should
FR-17	UI	Disclaimer on every result	Must
FR-18	History	Save past checks per patient and export a result as PDF	Could

Non-functional requirements

A full check must return in under 5 seconds with zero uncited explanation sentences.

ID	Quality	Requirement
NFR-01	Speed	Full check under 5 s; severity and score shown under 1 s, explanation streams in after
NFR-02	Safety	Recall of 0.90 or higher on Major interactions; a missed Major is the worst error
NFR-03	Safety	When severity is unknown and the ML confidence is below 0.6, show "Unknown — consult a pharmacist", never Low
NFR-04	Faithfulness	100% of explanation sentences cite a retrieved label passage
NFR-05	Transparency	Every point in the score traces to one visible rule
NFR-06	Privacy	Synthetic data only; no real patient data stored or sent to the LLM
NFR-07	Reliability	Cached RxNorm and label data so the demo runs if an external API is down
NFR-08	Usability	A first-time user completes a check with no instructions; works on phone and laptop
NFR-09	Cost	Free tiers only: free LLM API (Gemini or Groq) or local model, local embeddings, local databases

ID	Quality	Requirement
NFR-10	Maintainability	LLM provider switchable in config; unit tests for the rules engine; one-command setup (Docker Compose)

Datasets and APIs

All six sources are free for academic use; DDInter is the ground truth for severity and Synthea supplies every patient.

Source	What it provides	Approx. size	Used for	Access
RxNorm / RxNav API (NLM)	Standard drug names and IDs, brand → ingredient mapping, spelling suggestions	All US-marketed drugs	Normaliser (FR-02, FR-03)	Free REST API, no key
DDInter	Drug-drug interaction pairs labelled Major / Moderate / Minor	About 1,800 drugs, about 240,000 pairs	Base severity (FR-04); labels for ML training	Free CSV download, academic licence
openFDA drug labels	FDA label sections: drug interactions, warnings, contraindications, pharmacology	Tens of thousands of labels (JSON)	RAG knowledge base (FR-11); condition cautions and kidney/liver flags	Free API and bulk download
PubChem	Chemical structure (SMILES) per drug	100M+ compounds; about 1,800 needed	ML features (molecular fingerprints)	Free PUG REST API
Synthea	Synthetic patients in FHIR R4: age, conditions, labs, medications	Generate 500–1,000 patients	Patient context (FR-06)	Open-source generator

Source	What it provides	Approx. size	Used for	Access
Curated rules file (built in this project)	Per-drug flags: kidney-cleared, liver-metabolised, condition cautions	Top 150 common drugs	Rules engine (FR-10)	Extracted from openFDA labels by LLM, checked by hand

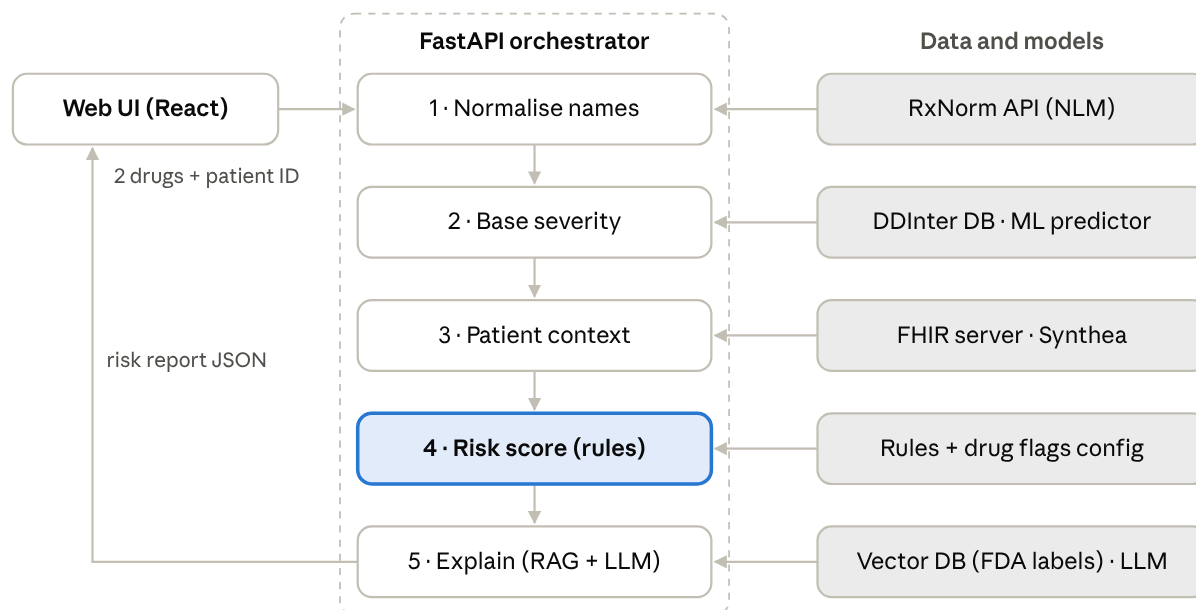
Two data notes.

- Synthea records do not reliably include alcohol use; a small script adds a drinks-per-day observation to each generated patient, and this is stated in the report.
- NLM retired its free drug-interaction API in January 2024, which is why severity comes from DDInter rather than RxNav.

Sizes are approximate from memory; confirm them on each source page before submission.

System architecture

One FastAPI orchestrator runs five fixed steps in order, each reading from its own data source, and returns a single JSON report to the UI.

RxGuard architecture: one request, five steps

request flow · 5 steps, 5 data sources

Step 4 is the core of the project: it is the only place patient data changes the result, and it is plain rules, so every point is explainable.

Offline pipelines (run once, before the app starts)

- 1. Interaction store:** DDInter CSVs → cleaned, merged by RxNorm ID → SQLite table of pairs and severities.
- 2. ML training:** PubChem SMILES → Morgan fingerprints for each drug → pair features → 3-class severity classifier trained on DDInter labels → saved model file.
- 3. RAG index:** openFDA labels for the covered drugs → split by section (interactions, warnings, contraindications) → embedded → stored in the vector DB with drug and section metadata.
- 4. Rules config:** LLM extracts kidney/liver flags and condition cautions from labels → hand-checked → YAML file.
- 5. Patients:** Synthea generates 500–1,000 FHIR patients → alcohol observation added → loaded into a HAPI FHIR server.

Tech stack

Everything runs on a laptop with free tools; only the LLM call leaves the machine.

Layer	Choice	Why
Frontend	React + Vite, Tailwind CSS, Recharts	Polished, responsive UI; gauge and breakdown charts
Backend	Python, FastAPI	Async API, easy to stream the explanation
Orchestration	LangChain (or plain Python pipeline)	Clear step-by-step flow; swappable LLM
LLM	Gemini or Groq free API; Ollama (local) as fallback	Free; provider set in one config line
Embeddings	sentence-transformers (all-MiniLM-L6-v2)	Free, runs on CPU
Vector DB	ChromaDB	Local, simple, stores metadata for citations
ML	scikit-learn / XGBoost, RDKit for fingerprints	Standard, fast on CPU, explainable feature importance
Interaction store	SQLite	Zero setup for about 240,000 pairs
Patient records	Synthea + HAPI FHIR server (Docker)	Real healthcare data standard (FHIR R4)
Testing	pytest; RAGAS for explanation faithfulness	Rules unit tests; measurable RAG quality
Deployment	Docker Compose; frontend on Vercel for the portfolio link	One-command setup; shareable demo

Evaluation plan

Each component is measured separately, and the whole system is tested on 50 hand-built patient scenarios.

Component	Test	Metric	Target
Normaliser	200 names: brands, generics, misspellings	Top-1 accuracy	95% or higher
ML severity predictor	20% of DDInter pairs held out, split by drug so test drugs are unseen	Macro F1; recall on Major	F1 0.70+; Major recall 0.90+
ML vs baseline	Same split, majority-class and logistic-regression baselines	F1 improvement	Clearly above both
RAG retrieval	100 questions with the correct label section marked	Recall@5	0.85 or higher
Explanation	100 generated explanations	Faithfulness (RAGAS) and % sentences with a valid citation	0.90+; 100%
Rules engine	Unit tests for every rule and boundary (age 64 vs 65, eGFR 59 vs 60)	Pass rate	100%
End to end	50 scenarios (drug pair + patient) with expected risk band, checked against labels	Band accuracy; High cases missed	85%+; zero missed
Speed	50 scenarios	Median and 95th-percentile response time	Under 5 s at the 95th percentile

The headline interview result is the side-by-side: the same pair scoring differently for two patients, with every point traced to a rule.

Timeline, risks and limitations

The plan assumes a 12-week semester, with a working end-to-end demo by week 8.

Weeks	Milestone	Deliverable
1-2	Data setup	DDInter in SQLite, RxNorm client, 500 Synthea patients in HAPI FHIR
3-4	ML predictor	Fingerprint features, trained classifier, baseline comparison
5	Rules engine	YAML rules for 150 drugs, unit tests passing
6-7	RAG explainer	openFDA index in ChromaDB, cited explanations, citation check
8	End-to-end backend	FastAPI pipeline returning the full JSON report
9-10	UI	Gauge, breakdown, body map, two-patient compare
11	Evaluation	All metrics in the evaluation table, 50 scenarios scored
12	Final	Report, README with architecture, demo video, deployed link

Risks

Risk	Impact	Mitigation
ML misses a Major interaction	Unsafe result	Tune threshold for Major recall; low confidence shows "Unknown — consult a pharmacist"
LLM invents a mechanism	Wrong explanation	Answer only from retrieved passages; drop uncited sentences
Rule weights are arbitrary	Hard to defend	Tune on scenarios; show breakdown; state it as a design choice
External API down during demo	Demo fails	Cache RxNorm and label data locally
DDInter licence or download changes	No ground truth	Keep a local copy; TWOSIDES as backup

Limitations

- Synthetic patients only; results are not validated on real clinical outcomes.
- Two drugs per check; real patients often take five or more.
- Rules cover about 150 common drugs; others get base severity only.
- Not a medical device; decision support only.